

Astro Monitor

Sistema Inteligente de Monitoramento de Missão Espacial

Integrante	RM
Aelton	RM 573694
Victor Mantovani	RM 570608
Bruno	RM 572073
Michelly	RM 573625
Maria Eduarda	RM 572267

Equipe: {Dev}Lounge | FIAP — Global Solution 2026

Repositório: github.com/DevLounge-FIAP/gs-fiap-astro-monitor

Video: youtube.com/watch?v=NpEUCEKVi34

1. Introdução e Contexto do Problema

As missões espaciais modernas dependem de sistemas inteligentes de monitoramento contínuo para garantir a segurança operacional e a eficiência dos recursos. Em ambientes onde a comunicação é limitada e as condições são extremas, os dados gerados pelos sensores tornam-se a principal fonte de informação para tomada de decisão.

O Astro Monitor foi desenvolvido para simular o sistema de controle e monitoramento de uma missão espacial experimental em Marte. A missão opera em ciclos de 5 minutos durante 500 ciclos totais (aproximadamente 41 horas), coletando telemetria de energia, suporte à vida, integridade estrutural e condições ambientais.

O sistema ingere dados de um arquivo CSV gerado com variações realistas, incluindo uma anomalia intencional entre os ciclos 250 e 300: uma tempestade de areia que reduz a geração solar a quase zero, eleva a radiação e degrada levemente a integridade estrutural. Essa inconsistência proposital testa a capacidade de diagnóstico do motor de regras.

O objetivo central é identificar situações críticas automaticamente, gerar alertas priorizados e prever o comportamento da bateria para antecipar decisões operacionais antes que se tornem emergências.

2. Estruturas de Dados Utilizadas

A organização dos dados da missão seguiu os requisitos do enunciado, aplicando cada estrutura onde ela oferece a melhor relação entre clareza e eficiência:

Estrutura	Onde é usada	Justificativa
Dicionário (dict)	self.modulos, self.sistemas, self.sensores	Acesso $O(1)$ por chave — localiza módulos e sensores sem percorrer listas
Lista (list)	historico_geracao, historico_leituras	Armazena séries temporais para cálculos de média e regressão linear
Fila (deque — FIFO)	fila_de_alertas em CentralDeAlertas	append e popleft em $O(1)$ — alertas processados na ordem de chegada
Pilha (list — LIFO)	log_eventos_criticos em Missao_espacial	append e acesso por $[-1]$ — histórico dos eventos críticos mais recentes
Matriz (lista de listas)	gerar_matriz_telemetria()	Organiza ciclo, geração, consumo e bateria sem dependência de numpy
Hierarquia de objetos	Missao_espacial > Modulo > Sistema/Sensor	Representa a estrutura física da missão em três níveis de abstração

A escolha do deque para a fila de alertas merece destaque: enquanto uma lista comum tem custo $O(n)$ para remoção no início, o deque garante $O(1)$ em ambas as extremidades. Com centenas de alertas gerados por ciclo, essa diferença impacta diretamente a performance do sistema.

A hierarquia Missao_espacial → Modulo → Sistema/Sensor espelha a estrutura física real de uma base marciana: a missão agrega módulos (energia, suporte à vida, comunicação), cada módulo contém sistemas de geração ou armazenamento e sensores de medição acoplados.

3. Lógica de Diagnóstico e Regras Booleanas

O motor de regras foi implementado na classe MotorDeRegras em src/rules/regras.py. A cada ciclo de telemetria, o método analisar_status_geral() avalia cinco grupos de regras e enfileira alertas na CentralDeAlertas conforme as condições detectadas.

Expressão booleana principal do diagnóstico:

```
STATUS_CRITICO = (bateria < 30 AND (solar + eolico) < demanda)
                  OR (radiacao > 1 AND solar < 3000)
                  OR (o2 < 19 AND temp_int < 15)
                  OR (integridade == 0) OR (integridade < 70)
```

Quando STATUS_CRITICO = True, o sistema enfileira um alerta CRÍTICO e bloqueia o alerta NORMAL. Quando todos os parâmetros estão dentro das faixas seguras, confirma status NORMAL.

Regra	Condição	Severidade	Recomendação
Energia insuficiente	bateria < 30 AND geracao < demanda	CRITICO	Desligar módulos não essenciais
Risco de energia	bateria < 50 AND geracao < demanda	ALERTA	Desligar módulos de baixa criticidade
Painéis comprometidos	radiacao > 1 AND solar < 3000	CRITICO	Verificar placas solares
Sem geração eólica	NOT eolico	ALERTA	Verificar turbinas eólicas
Risco à vida (combinado)	o2 < 19 AND temp_int < 15	CRITICO	Verificar suporte à vida urgente
Risco à vida (isolado)	o2 < 19 OR temp_int < 15	ALERTA	Verificar suporte à vida
Integridade crítica	integridade < 70	CRITICO	Enviar reparo imediato
Integridade degradada	integridade < 90	ALERTA	Verificar integridade
Falha de sensor	temp_int > 1000 OR bateria < 0	CRITICO	Reiniciar sensores
Módulos estáveis	todos dentro das faixas	NORMAL	Continuar operação padrão

O uso da flag status_critico garante que o alerta NORMAL só seja emitido quando nenhuma das regras anteriores foi ativada, evitando contradições como emitir NORMAL e CRITICO no mesmo ciclo.

4. Dados Simulados e Anomalia Intencional

Os dados foram gerados pelo script `data/gerar_dados.py`, produzindo 500 ciclos temporais no arquivo `data/dados.csv`. Cada linha representa um ciclo de 5 minutos com as seguintes variáveis:

Variável	Unidade	Descrição
Geracao_Solar_W	Watts	Geração dos painéis solares — varia com ciclo diurno (função seno)
Geracao_Eolica_W	Watts	Geração das turbinas eólicas — variação aleatória dentro de faixas
Demanda_Global_W	Watts	Consumo total da missão — aumenta durante a tempestade
Nivel_Bateria_Pct	%	Nível de carga da bateria — calculado pelo saldo energético acumulado
O2_Pct	%	Percentual de oxigênio interno — varia levemente (18% a 23%)
Temp_Int_C	Celsius	Temperatura interna — cai levemente durante a tempestade
Temp_Ext_C	Celsius	Temperatura externa — entre -100°C e -30°C
Radiacao_mSv	mSv	Nível de radiação — sobe entre 2.0 e 5.0 mSv durante a tempestade
Integridade_Pct	%	Integridade estrutural — sofre dano leve nos ciclos 250 a 300

Anomalia intencional — Tempestade de Areia (ciclos 250 a 300):

Entre os ciclos 250 e 300, o sistema simula uma tempestade de areia marciana. Durante esse período, a geração solar cai para entre 0 e 50 W (painéis cobertos de poeira), a radiação sobe para entre 2.0 e 5.0 mSv, os ventos fortes aumentam a geração eólica para entre 2500 e 3500 W, a demanda global sobe (maior consumo de aquecimento), a temperatura interna cai levemente e a integridade estrutural sofre dano progressivo a cada ciclo.

Essa inconsistência proposital testa se o motor de regras consegue identificar corretamente a discrepância entre radiação elevada e geração solar baixa, gerando o alerta CRITICO de painéis comprometidos.

Módulos críticos monitorados:

ID	Tipo	Criticidade	Sistemas / Sensores
ENE-01	Módulo Energético	5 (máx)	Solar, Eólico, Bateria + Sensor de Demanda
SUP-01	Suporte à Vida	5 (máx)	Sensores de O2, Temp. Interna, Temp. Externa, Radiação, Integridade
COM-01	Comunicação	4	Transmissão de dados para a Terra
HAB-01	Habitat	3	Alojamento da tripulação
LAB-01	Laboratório	3	Pesquisa científica
ARM-01	Armazenamento	2	Estoque de suprimentos

5. Análise e Previsão de Dados

O sistema implementa dois módulos de análise em `src/forecast/`: a `AnaliseEnergetica`, que calcula estatísticas descritivas sobre os ciclos de telemetria, e o `ModeloPrevisaoBateria`, que aplica regressão linear simples para estimar o comportamento futuro da bateria.

5.1 Análise Energética Descritiva

A classe `AnaliseEnergetica` percorre a matriz de telemetria e calcula, sem bibliotecas externas, médias, mínimos e máximos de geração, consumo e bateria. Também identifica o pior ciclo registrado (maior déficit entre consumo e geração) e classifica a eficiência geral da missão por faixas de cobertura energética.

5.2 Regressão Linear para Previsão da Bateria

O `ModeloPrevisaoBateria` implementa regressão linear simples pelo método dos mínimos quadrados (least-squares) utilizando apenas operações aritméticas básicas, sem `numpy` ou `scikit-learn`. As fórmulas utilizadas são:

$$m = (n \times \text{SUM}(xy) - \text{SUM}(x) \times \text{SUM}(y)) / (n \times \text{SUM}(x^2) - \text{SUM}(x)^2)$$
$$b = (\text{SUM}(y) - m \times \text{SUM}(x)) / n$$

Onde `x` representa os índices dos ciclos e `y` os níveis de bateria correspondentes. O modelo calcula a inclinação `m` (taxa de variação por ciclo) e a interceptação `b` (valor inicial estimado).

Como a previsão influencia o sistema:

Após o treinamento com os 500 ciclos do CSV, o modelo projeta o nível da bateria para +10, +50 e +100 ciclos futuros. Se a inclinação `m` for negativa, o sistema calcula o ciclo estimado de colapso (quando a bateria atingiria 20%) e orienta o operador a acionar medidas preventivas antecipadamente. Essa previsão é exibida com clareza no menu e influencia diretamente as recomendações geradas.

Ciclo projetado	Nível de bateria	Recomendação associada
Ciclo 510	70.2%	Situação controlada
Ciclo 550	64.1%	Atenção recomendada
Ciclo 600	57.8%	Iniciar economia de energia
Ciclo 1200 (colapso)	20.0%	Limite crítico — ação imediata

6. Decisões Técnicas e Organização do Código

O projeto foi estruturado em módulos com responsabilidades bem definidas, seguindo o princípio de separação de responsabilidades:

Arquivo	Responsabilidade
<code>src/sistema.py</code>	Orquestrador principal com menu interativo e fluxo de execução
<code>src/modules/modulos.py</code>	Classes <code>Missao_espacial</code> , <code>Modulo</code> , <code>Sistema</code> , <code>Sensores</code> e todas as subclasses

src/rules/regras.py	Motor de regras booleanas — MotorDeRegras com analisar_status_geral()
src/rules/alertas.py	CentralDeAlertas com fila deque para gerenciamento de alertas
src/forecast/previsao.py	ModeloPrevisaoBateria com regressão linear manual
src/forecast/analise_energetica.py	AnaliseEnergetica com estatísticas descritivas por ciclo
data/gerar_dados.py	Gerador dos 500 ciclos de telemetria com anomalia da tempestade

A decisão de usar injeção de dependência no MotorDeRegras — recebendo a missão e a central de alertas prontos no construtor — facilita o teste isolado de cada componente e mantém o acoplamento baixo entre os módulos.

O uso de herança nas classes de sensores (SensorO2, SensorRadicao, SensorTemperaturaInterna, entre outras) a partir da classe base Sensores demonstra aplicação prática de orientação a objetos: cada subclasse especializa apenas o tipo e a função do sensor, reaproveitando toda a lógica de histórico e leitura da classe pai.

7. Conclusão

O Astro Monitor demonstrou como conceitos fundamentais de computação podem ser aplicados em um cenário realista e crítico. A integração entre estruturas de dados, lógica booleana e análise preditiva resultou em um sistema capaz de interpretar telemetria, classificar situações operacionais e antecipar riscos de forma automatizada.

A escolha de estruturas adequadas — dicionários para acesso $O(1)$, deque para fila eficiente, lista como pilha de eventos — demonstrou na prática como decisões de engenharia impactam a performance e a clareza do código. A implementação da regressão linear do zero, sem bibliotecas, reforçou a compreensão dos fundamentos matemáticos antes de abstrair para ferramentas prontas.

A simulação da tempestade de areia como anomalia intencional evidenciou como dados inconsistentes testam a robustez de qualquer motor de regras — e como a combinação de operadores lógicos AND, OR e NOT permite construir diagnósticos precisos mesmo em cenários complexos. Essa experiência conecta diretamente o aprendizado acadêmico ao que a indústria de tecnologia exige: transformar dados em decisões confiáveis.